An Incumbent-Protected Component-Local Heuristic for Minimum Weighted Feedback Arc Set on Sparse Digraphs

Soroush Vahidi[1*]

[1*]Department of Computer Science, New Jersey Institute of Technology, Newark, NJ 07102, USA

ORCID: 0000-0003-1934-6282.

Corresponding author: sv96@njit.edu

Abstract

Purpose: The minimum weighted feedback arc set problem seeks a minimum-weight arc set whose removal makes a directed graph acyclic, equivalently a vertex ordering of minimum backward-arc weight. It is NP-hard and arises in precedence aggregation, scheduling, and ranking from directed evidence. We target sparse nonnegative weighted digraphs, where exact methods do not scale broadly and dense ordering heuristics are not always well aligned.

Methods: We propose an incumbent-protected, component-local destroy-and-repair heuristic, denoted incumbent-protected SCC neighborhood search (IPSNS). IPSNS scores strongly connected components by backward-weight contribution, perturbs a randomly selected component neighborhood, repairs it with local-ratio reduction and heavy-first add-back, and accepts only strict global improvements. IPSNS refines the better of two attributed seeds, a local-ratio topological add-back seed and a weighted minimal/stable-style seed, and is evaluated against exact, mixed-integer, external-tool, and dense-transfer baselines.

Results: Among the evaluated methods, IPSNS attains the minimum observed backward weight on 96 of 97 standard nonnegative sparse instances, with ties credited to every tied method (14 strict improvements, 83 ties, 0 regressions versus the better seed). On a 57-instance exact-validation subset, the mean optimum-normalized gap is 0.0031%. Repeated-run per-instance medians over 20 runs on a 93-instance common subset give 38 wins, 55 ties, and 0 losses against DRMacIver/FAS, with a mean comparator-normalized reduction of 21.60%. On 50 dense LOLIB instances, the comparator was favored on 45.

Conclusion: Incumbent-protected, component-local refinement is an effective sparse-digraph algorithm-engineering strategy among the evaluated methods,

though it does not dominate dense complete-ordering benchmarks, where matrix-based heuristics remain preferable.

Keywords: Feedback arc set, Graph algorithms, Combinatorial optimization, Heuristic search, Strongly connected components, Algorithm engineering

1 Introduction

The minimum weighted feedback arc set (MWFAS) problem asks for a minimum-weight set of arcs whose removal makes a directed graph acyclic; equivalently, it asks for a vertex ordering with minimum total backward-arc weight. The problem is NP-hard [1] and arises whenever directed pairwise relations contain inconsistent cycles, including precedence aggregation, dependency repair, scheduling, and ranking from directed evidence [2, 3]. In weighted settings, not all violations are interchangeable: reversing a high-weight precedence relation is more costly than reversing a weak one, so useful heuristics must react to both graph structure and arc-weight asymmetry.

This paper focuses on *sparse nonnegative weighted digraphs*. That regime differs structurally from dense linear-ordering or tournament instances. In many sparse graphs, large regions are already acyclic or only weakly cyclic, while the hard part of the objective is concentrated in a smaller set of strongly connected components (SCCs). Dense complete ordering instances behave differently: every ordered pair carries weight, the cyclic core effectively spans the whole graph, and matrix-based pairwise-ordering heuristics are naturally aligned with the input model. A method that is strong on sparse digraphs therefore need not be strongest on dense ordering benchmarks.

This creates a practical gap. Exact formulations provide certification but do not scale broadly enough to serve as the main tool on the sparse benchmark studied here [4]. Classical greedy heuristics remain fast and interpretable, yet they may spend effort too globally or may not exploit the fact that residual backward weight is often localized after a strong constructive pass. Earlier local-ratio and weighted minimal/stable constructions provide useful seeds, but they leave open whether a *protected* SCC-local refinement stage can improve solutions consistently without ever returning a worse answer than the best available seed.

The central contribution is an *incumbent-protected, component-local heuristic*. It begins from the better of two attributed seeds, scores nontrivial strongly connected components by their current backward-weight contribution, and applies component-local destroy-and-repair moves inside fixed original-graph neighborhoods. The destroy phase reactivates heavy removed internal arcs and removes light active internal arcs using deterministic prefixes; the repair phase reruns local-ratio reduction and heavy-first add-back only inside the selected component neighborhood. A candidate is accepted only if the repaired global ordering is feasible and strictly improves backward weight; otherwise the incumbent is restored exactly. We refer to this heuristic

as incumbent-protected SCC neighborhood search (IPSNS); it is therefore stochastic through score-weighted component selection but monotone in returned objective value.

The two seeds are supporting components rather than coequal contributions. LR-TA (local-ratio topological add-back) is an engineered sparse-graph seed derived from the directed local-ratio lineage of Demetrescu and Finocchi [5] and connected to earlier author work on ranking-oriented MWFAS [6]. The WMSF-style seed is an adapted implementation of the weighted minimal/stable feedback arc set line of Cavallaro and Cutello [7]. The present paper does not claim those foundations as new; it uses them as complementary entry points whose better incumbent IPSNS then protects and selectively improves.

The empirical message is intentionally narrow. On the primary sparse comparison, IPSNS attains the minimum observed backward weight among the evaluated methods on 96 of 97 standard nonnegative instances, with ties credited to every tied method (14 strict wins). A paired 97-instance comparison against the better seed yields 14 strict improvements, 83 ties, and 0 regressions. On the 57-instance exact-validation subset, the mean optimum-normalized gap is 0.0031%. On the 93-instance common sparse subset, repeated-run per-instance medians over 20 runs yield 38 wins, 55 ties, and 0 losses for IPSNS against the evaluated DRMacIver/FAS executable. A single-run dense LOLIB transfer study shows that the selected complete-ordering benchmark and evaluated implementations favored DRMacIver/FAS on 45 of 50 instances. The paper's claim is therefore that protected SCC-local refinement is an effective sparse-graph algorithm-engineering strategy among the evaluated methods, not that it solves every MWFAS regime equally well.

The contributions are:

- **Algorithmic contribution.** We introduce IPSNS, an incumbent-protected, component-local destroy-and-repair heuristic for sparse nonnegative weighted digraphs. The contribution lies in the problem-specific integration of fixed component neighborhoods, contribution-based component selection, two-sided perturbation, component-restricted repair, and strict global-improvement acceptance.
- **Integrated framework.** IPSNS is evaluated together with two explicitly attributed constructive seeds. The first is an engineered seed derived from prior local-ratio work, denoted LR-TA. The second is adapted from Cavallaro and Cutello's weighted minimal/stable pipeline and is denoted WMSF-style.
- **Supporting analysis.** We formalize implementation-level properties for seed feasibility, one-pass add-back minimality, and IPSNS incumbent non-worsening, together with concise complexity summaries. These are supporting guarantees for the implemented framework rather than new approximation results.
- **Computational evidence.** We report sparse benchmark comparisons, exact and MIP validation, ablation and holdout-supported parameter checks, repeated-run robustness against an external executable with run-to-run variation, and a dense LOLIB scope-boundary study.

The remainder of the paper positions the work, defines the objective, presents the framework, summarizes the experimental design, reports the results, discusses the sparse and dense regimes, and concludes.

2 Related work

This section positions the present heuristic within three adjacent literatures: graph algorithms for cycle breaking, experimental algorithmics and heuristic design for combinatorial optimization, and the specific minimal/stable and SCC-aware feedback-set lineage that motivates the proposed component-local refinement step.

2.1 Feedback arc set algorithms

Minimum feedback arc set sits between graph-theoretic cycle breaking and ordering optimization. Classical formulations treat it as deleting a minimum-weight arc set to obtain an acyclic digraph or, equivalently, as finding an ordering of minimum backward weight [2, 8]. The decision version is NP-hard [1], and exact methods remain important mainly as certification tools on restricted instance classes. Modern exact formulations connect MWFAS to broader combinatorial-optimization machinery [4], but they do not eliminate the practical need for heuristics on larger sparse graphs – the territory of experimental algorithmics and algorithm engineering that this paper occupies.

Approximation and reduction-based work provides methodological context rather than the main claim of this paper. The general local-ratio framework of [9] explains why iterative weight reductions can preserve improvement structure in weighted optimization. For directed feedback problems, Demetrescu and Finocchi [5] show how cycle-based local-ratio reductions can be used algorithmically; that work is the main conceptual ancestor of the LR-TA seed used here. Earlier directed-feedback approximation results [10] and more recent localization-oriented engineering work [11] place the present heuristic study within a broader feedback-set literature.

Classical heuristic baselines remain relevant because many practical implementations still inherit their design logic. Greedy score-based ordering rules in the Eades family [12, 13] are fast and interpretable, but they act mainly through global score differences rather than targeted treatment of residual cyclic subgraphs. Comparative studies such as [14] show that SCC-aware decomposition often improves over whole-graph greedy ranking, which is directly relevant to the local repair strategy of IPSNS. At larger scales, deployment-oriented engineering becomes central [15].

2.2 Ordering and ranking formulations

The ordering interpretation of MWFAS connects the problem to ranking and linear ordering. In dense pairwise-comparison settings, the natural input is often a nonnegative matrix W whose entry W_{ij} expresses evidence that item i should precede item j, and the objective is to maximize total forward weight or minimize total backward weight [3, 16]. That viewpoint is especially natural for complete tournaments and dense linear-ordering benchmarks. In sparse digraphs, many ordered pairs are

absent altogether, so the problem is more naturally expressed as cycle breaking in an incomplete directed graph.

This distinction matters for baseline selection and claim scope. The python-igraph interface [17] provides an accessible graph-library comparator. The DRMacIver/FAS tool [18], by contrast, is an external matrix-based ordering heuristic whose documented input model is pairwise-ordering data rather than sparse graph analysis. The evaluated executable exhibits run-to-run variation under internal initialization, so it is treated empirically as stochastic here. This makes it both a useful cross-paradigm comparator on sparse instances and an especially appropriate comparator on dense LOLIB instances, where the matrix-based formulation aligns naturally with the benchmark structure.

The dense linear-ordering literature provides the right background for the LOLIB transfer test. LOLIB was established as a benchmark family for complete weighted ordering problems rather than sparse general digraphs [19, 20]. Tournament and dense-ordering heuristics therefore form a separate comparison class, including weighted local-search methods [21] and more recent dense-ordering work such as [22]. The present paper uses LOLIB to mark a scope boundary rather than to claim superiority in that dense regime.

2.3 Minimal/stable FAS and SCC-based heuristics

SCC structure is a recurring theme in practical feedback arc heuristics because cycles are localized there. Decomposition along SCC boundaries is therefore a natural way to avoid spending effort on regions that are already acyclic. The present work uses that insight twice: in the attributed seed constructions and again in the IPSNS refinement layer, where only SCCs with positive incumbent backward contribution are eligible for repair.

The second seed in the framework comes from the minimal/stable feedback arc set line of Cavallaro and collaborators. Cavallaro and Cutello [7] study weighted minimal-and-stable feedback arc sets, extending earlier minimal/stable constructions [23]. That lineage matters here because it provides a complementary constructive bias to local-ratio seeding and because it distinguishes *minimal* from *minimum-weight*: an inclusion-minimal feedback arc set is one from which no arc can be restored while preserving acyclicity, but it need not minimize total weight.

This distinction is easy to blur in heuristic writing, so the paper keeps it explicit. LR-TA and the WMSF-style seed both use acyclicity-preserving add-back, and IPSNS inherits the same add-back logic inside SCC-restricted repair. What matters scientifically is not a claim of exact solution, but that these routines provide feasible and complementary seeds for later incumbent-protected refinement.

2.4 Relationship to preliminary author work and present contribution

The public preprint of Vahidi and Koutis [6] introduced the ranking-oriented framing of MWFAS together with an earlier local-ratio-based pipeline built around cycle peeling, feasible reinsertion, and topological extraction. The present manuscript does not

claim that foundation as new. The LR-TA seed descends from that earlier pipeline and is attributed accordingly. The WMSF-style seed is likewise not presented as a new contribution of this paper; it is an adapted implementation of the weighted minimal/stable line of Cavallaro and Cutello [7].

What is new here is the IPSNS-centered framework and the accompanying computational study: dual-seed initialization, supporting feasibility and monotonicity analysis, exact and MIP validation, sparse benchmark comparisons, repeated-run analysis against the evaluated DRMacIver/FAS executable, holdout-supported defaults, and a dense LOLIB scope study. The contribution is therefore a sparse-digraph algorithm-engineering paper centered on incumbent-protected SCC-local refinement, with inherited components attributed rather than rebranded.

3 Problem definition

Let $G = (V, A, w)$ be a directed graph with vertex set V, arc set $A \subseteq V \times V$, and nonnegative arc weights $w : A \to \mathbb{R}_{\geq 0}$. A feedback arc set is a set $F \subseteq A$ such that removing F makes the graph acyclic. The minimum weighted feedback arc set problem asks for a minimum-weight feedback arc set, i.e., a set F minimizing $\sum_{a \in F} w(a)$.

An ordering π assigns each vertex a distinct position. An arc (u, v) is backward under π if u appears after v. The ordering-induced backward weight is

$$\mathrm{bw}(\pi) = \sum_{(u,v) \in A:\ \pi(u) > \pi(v)} w(u, v). \tag{1}$$

For any ordering π, the set of backward arcs is a feedback arc set: all remaining forward arcs point from earlier to later positions and therefore form an acyclic graph. Conversely, every acyclic subgraph admits a topological order. The weighted feedback arc set problem can therefore be treated as the problem of finding an ordering with minimum backward weight [1, 2].

Many constructive heuristics in this study first build an acyclic *active* subgraph $H = (V, A \setminus F)$ by deleting a set $F \subseteq A$, and then extract a linear extension π of H. For such a pair (F, π), define

$$B_\pi = \{(u, v) \in A :\ \pi(v) < \pi(u)\}. \tag{2}$$

Every active arc is forward under π, so $B_\pi \subseteq F$. For nonnegative weights,

$$\mathrm{bw}(\pi) = w(B_\pi) \leq w(F). \tag{3}$$

Equality need not hold: topological extraction is generally *not* objective-neutral, because different linear extensions of the same active DAG can yield different values of $\mathrm{bw}(\pi)$. The implementation therefore reports the ordering objective recomputed from the returned ranking, not the removed-set weight $w(F)$ alone. Online Resource 1 records the detailed extraction discussion and calibration checks.

Dense linear ordering problem benchmarks are often stated in terms of a complete weight matrix C, where an ordering $\pi = (\pi_1, \ldots, \pi_n)$ is evaluated by the forward weight. In that notation,

$$\mathrm{fw}_C(\pi) = \sum_{1 \le i < j \le n} C_{\pi_i,\pi_j}, \qquad \mathrm{bw}_C(\pi) = \sum_{1 \le i < j \le n} C_{\pi_j,\pi_i} = W_C - \mathrm{fw}_C(\pi), \tag{4}$$

where $W_C = \sum_{i \neq j} C_{ij}$ is the total off-diagonal weight. Thus maximizing forward weight on a complete dense linear-ordering instance is equivalent to minimizing backward weight after converting the matrix to a complete weighted digraph. LOLIB results are reported as a transfer test. Those instances are dense complete ordering problems rather than sparse directed graph benchmarks.

4 Proposed methodology

The framework has three components: two attributed constructive seeds and one refinement layer. IPSNS is the central method. LR-TA and the WMSF-style routine are included because they provide complementary feasible starting points and because the IPSNS incumbent-protection guarantee is interpreted relative to the better of those two seeds. Online Resource 1 provides a compact framework schematic.

All methods operate on the aggregated weighted digraph of Section 3 and optimize backward weight.

4.1 Local-ratio topological add-back

LR-TA is an engineered seed derived from the directed local-ratio lineage of [5] and from earlier author work [6]. It is not presented as a new approximation result. Its role in the present paper is to provide a strong sparse-graph seed whose output can later be improved, or preserved, by IPSNS.

Phase I repeatedly finds any directed cycle in the active graph and subtracts the minimum residual weight on that cycle from every arc in the cycle. Let $W(e)$ denote the mutable residual weight of active arc e and $W_0(e)$ the original aggregated weight. If C is the detected cycle, the reduction amount is

$$\varepsilon(C) = \min_{e \in C} W(e). \tag{5}$$

At least one arc becomes tight and is removed from the active graph. Phase I ends when the active graph is acyclic.

Phase II performs one-pass heavy-first add-back. Removed arcs are processed in nonincreasing order of original weight. A removed arc is restored immediately if the current topological order already places it forward; otherwise it is restored only when a reachability test shows that reinsertion preserves acyclicity. Backward restorations trigger topological recomputation before later tests. The extracted topological order is deterministic under the chosen implementation rule, but it remains only one linear extension of the final DAG.

Algorithm 1 summarizes LR-TA.

The important point for the rest of the paper is that LR-TA delivers a feasible ordering together with a recorded removed set and a strong heavy-first repair stage.

Algorithm 1 LR-TA seed construction

Require: Weighted digraph $G = (V, A, w)$ with $w \geq 0$
Ensure: Ordering π_{LR}, removed-edge set F_{LR}
$F_{\mathrm{LR}} \leftarrow \emptyset$
while the active graph contains a directed cycle C **do**
  $\varepsilon \leftarrow \min_{e \in C} W[e]$
  Subtract ε on C; deactivate every newly tight active arc and add it to F_{LR}
end while
Compute deterministic topological order π of the active graph
for all $e = (u, v) \in F_{\mathrm{LR}}$ in nonincreasing order of $W_0[e]$ **do**
  if $\mathrm{rank}_\pi(u) < \mathrm{rank}_\pi(v)$ or $v \not\rightsquigarrow u$ in the current active graph **then**
    reactivate e; remove e from F_{LR}
    recompute π if the accepted arc is backward under the previous order
  end if
end for
return $\pi_{\mathrm{LR}}, F_{\mathrm{LR}}$

Extended implementation commentary, tolerance details, and proof expansions are deferred to Online Resource 1.

4.2 WMSF-style seed construction

The second seed is an adapted implementation of the weighted minimal/stable feedback arc set line of Cavallaro and Cutello [7]. It is carried as a complementary seed, not as a separate novel claim. The routine decomposes the graph into SCCs, processes each nontrivial SCC independently, removes arcs according to either an $L1$ or $L2$ weight-based ordering, then applies heavy-first add-back, stabilization, and a second add-back pass. When the full graph is one nontrivial SCC, both $L1$ and $L2$ are tried and the better ordering is retained.

This seed provides a different bias from LR-TA: LR-TA is driven by cycle reduction in an active sparse graph, whereas the WMSF-style seed is driven by ordered removals and subsequent minimization/stabilization. Taking the better of the two before refinement therefore gives IPSNS a stronger incumbent than either seed alone. Step-by-step propagation details and source-to-code mapping appear in Online Resource 1.

4.3 Incumbent-protected SCC-neighborhood search

IPSNS is the primary contribution of the paper. Standard ingredients such as SCC decomposition, rollback, add-back, or destroy-and-repair are not claimed as new in isolation. What is new is their integration for sparse weighted MWFAS: fixed original-graph SCC neighborhoods, contribution-based SCC scoring, weighted top-K selection, two-sided perturbation, SCC-restricted local-ratio repair, SCC-restricted heavy-first add-back, and strict incumbent protection.

IPSNS first computes the two seeds and chooses the better ordering as the initial incumbent. At each iteration, it scores each nontrivial original-graph SCC by the backward weight currently contributed by internal arcs under the incumbent order. Only SCCs with positive contribution are eligible. One SCC is then sampled from the top-K eligible SCCs using score-proportional random choice.

Within the selected SCC, IPSNS executes a destroy-and-repair move. The destroy stage temporarily reactivates a heavy fraction of currently removed internal arcs and removes a light fraction of currently active internal arcs. The repair stage reruns

Algorithm 2 IPSNS with incumbent protection

Require: Weighted digraph $G = (V, A, w)$, iteration budget T
Ensure: Ordering $\pi^\star$ with objective no worse than the best internal seed
Build WMSF-style seed π_{W} and LR-TA seed π_{LR}
$\pi^\star \leftarrow \arg\min\{\mathrm{bw}(\pi_{\mathrm{W}}), \mathrm{bw}(\pi_{\mathrm{LR}})\}$
for $t = 1, \ldots, T$ **do**
    Score each nontrivial SCC by its current backward contribution
    if all SCC scores are zero **then**
        break
    end if
    Sample one SCC from the top-K positive-score SCCs using weighted random choice
    Save the current SCC state; reactivate heavy removed edges; remove light active edges
    Apply SCC-restricted local-ratio repair and heavy-first add-back
    if the candidate SCC state is invalid **then**
        restore saved SCC state; **continue**
    end if
    Recompute the global ordering and candidate objective
    if the candidate objective is smaller than $\mathrm{bw}(\pi^\star)$ **then**
        accept the candidate and update $\pi^\star$
    else
        restore the previous incumbent exactly
    end if
end for
return $\pi^\star$

local-ratio cycle reduction and one-pass heavy-first add-back only on the chosen SCC neighborhood, with original weights restored on that neighborhood. All edits outside the neighborhood remain fixed.

The candidate is accepted only if it induces a valid global ordering and strictly decreases backward weight. Otherwise the pre-move state is restored exactly. IPSNS therefore never returns a worse solution than the better seed from which it started. IPSNS is stochastic through score-weighted SCC selection from the top-K positive-score SCCs. Conditional on the selected SCC and current state, the heavy-add-back and light-removal prefixes are deterministic under the specified tie-breaking rules: removed internal arcs are sorted by descending weight and a prefix of length $k_{\mathrm{add}} = \lfloor f_{\mathrm{add}}|F_\cap| \rfloor$ is reactivated; active internal arcs are sorted by ascending weight and a prefix of length $k_{\mathrm{rem}} = \lfloor f_{\mathrm{rem}}|A_\cap| \rfloor$ is removed. For small SCCs one or both counts may be zero. The method is reproducible for fixed input, parameter settings, and seed.

Algorithm 2 summarizes the driver.

The parameterization is interpreted conservatively in this paper. Reported defaults are holdout-supported engineering choices, not claims of universal optimality. Full parameter grids, additional implementation commentary, and supplementary controls are deferred to Online Resource 1.

4.4 Supporting correctness properties and complexity

The following properties formalize feasibility and monotonicity of the implemented framework; they are supporting guarantees rather than new approximation results. Detailed proofs are deferred to Online Resource 1 (§S4). Throughout, $n = |V|$, $m = |A|$, T is the IPSNS iteration budget, and for a selected SCC S we write $n_S = |V(S)|$ and $m_S = |A(S)|$.

Proposition 1 (LR-TA feasibility and progress) *Assume nonnegative original weights and the active-graph representation used in the implementation. Each Phase-I cycle reduction subtracts $\varepsilon(C)$ from every arc in a detected directed cycle C, so at least one active arc becomes tight and is deactivated. Phase I therefore terminates after finitely many iterations and leaves an acyclic active graph. Phase II restores a removed arc only when the current topological order already places it forward or when a reachability test certifies that restoration preserves acyclicity. The final active graph is acyclic and induces a feasible ordering for the objective of Section 3.*

Proposition 2 (One-pass add-back minimality) *Let $H_i = (V, A_i)$ be the active acyclic graph after Phase I and let $H_f = (V, A_f)$ denote the final active graph after one-pass add-back. After every candidate removed arc has been processed once and reinserted whenever insertion preserves acyclicity, the remaining removed set F_f is inclusion-minimal in the feedback-arc-set sense:*

$$\forall e \in F_f, \quad (V, A_f \cup \{e\}) \text{ is cyclic.} \tag{6}$$

Equivalently, for every $e \in F_f$, restoring e to A_f creates a directed cycle.

If inserting $e = (u, v)$ is rejected, a directed path $v \rightsquigarrow u$ already exists in the current active graph. Later accepted insertions only add arcs and cannot destroy that path, so e cannot become feasible later in the same pass. Heavy-first ordering therefore affects which inclusion-minimal feedback arc set is obtained, not whether inclusion-minimality holds. Inclusion-minimality does not imply minimum weight, and this statement applies to one-pass acyclicity-preserving add-back rather than to WMSF stabilization swaps.

Proposition 3 (Incumbent non-worsening invariant) *Let $\pi^{(0)}$ be the better of the LR-TA and WMSF-style seeds, and let $\pi^{(t)}$ denote the incumbent after iteration t of IPSNS. Then every accepted candidate is feasible, every infeasible or non-improving candidate is rolled back exactly, and*

$$\mathrm{bw}\left(\pi^{(t+1)}\right) \le \mathrm{bw}\left(\pi^{(t)}\right) \le \mathrm{bw}\left(\pi^{(0)}\right) \qquad \text{for all } t \ge 0. \tag{7}$$

The outer loop executes at most T iterations and may terminate earlier when every eligible SCC has zero backward contribution.

The proof logic is straightforward. LR-TA progresses because each detected cycle reduction tightens at least one active arc. One-pass add-back is valid because rejected arcs remain blocked by an already existing reverse path. IPSNS is monotone because every failed or non-improving move is rolled back exactly before the next iteration.

Complexity summary.

The implementation uses sparse edge-indexed storage, so space usage is $O(n + m)$. LR-TA Phase I performs repeated cycle detection in the active graph and Phase II performs one-pass add-back, giving the conservative bound

$$O\big(m(n+m)\big) + O\big(r \log r + r(n+m)\big), \tag{8}$$

where r is the number of removed arcs examined in add-back. The WMSF-style seed first decomposes the graph into SCCs and then performs ordered removals, add-back, and stabilization inside each nontrivial SCC, yielding the conservative aggregate

bound

$$O\left(n + m + \sum_{S} \left(m_S \log m_S + r_S(n_S + m_S)\right)\right), \tag{9}$$

with r_S the number of add-back candidates processed in SCC S. One IPSNS iteration scores SCCs in $O(m)$, selects one SCC, performs SCC-restricted destroy-and-repair, and then recomputes the global objective, so a conservative per-iteration bound is

$$O(m + s \log s + c_S(n_S + m_S) + r_S \log r_S + r_S(n_S + m_S)), \tag{10}$$

where s is the number of eligible SCCs and c_S is the number of restricted cycle-reduction rounds. These are conservative implementation-cost summaries; seed construction plus T IPSNS iterations compose the full practical cost. Expanded justification appears in Online Resource 1.

5 Experimental design

The experiments are designed to support a scoped claim about sparse nonnegative weighted digraphs, not a universal claim across all feedback arc set regimes. The sparse graph-benchmark collection provides the primary evaluation setting. Exact and MIP validation provide certified reference points on subsets where certification is feasible. The repeated-run study evaluates typical behavior against an external executable whose initialization varies across runs. LOLIB serves as a transfer test that marks the dense complete-ordering boundary. Tables 1 and 2 summarize the methods and studies; Table 3 summarizes benchmark structure.

5.1 Benchmarks and scope

The primary benchmark family comes from the public `graph-benchmarks` collection [25]. The input list contained 123 instance paths; duplicate instance names were collapsed by keeping the first listed path for each basename, yielding 105 unique sparse DIMACS digraph instances. The main benchmark tables restrict attention to the 97 standard instances whose aggregated arc weights are nonnegative. Eight instances are excluded from the standard set because they contain negative-weight arcs: `gerez`, `howard-max`, `k3_3`, `ku`, `peterson`, `peterson1`, `peterson2`, and `stg0`. This is a methodological scope restriction rather than a favorable filter: the local-ratio construction and the interpretation of incumbent protection used here are stated for nonnegative weights.

Each DIMACS file is read line by line; comment and problem lines are ignored. Parallel arcs with the same ordered pair are aggregated by summing weights. Self-loops, if present, are retained. Isolated vertices appear only when referenced by arcs. Malformed arc records are skipped. Full preprocessing rules appear in Online Resource 1 (§S8).

Three benchmark subsets play supporting roles. First, the exact-validation subset consists of the 57 standard nonnegative instances with $0 < n \leq 20$, on which bitmask dynamic programming can certify the optimum. Second, a medium-scale subset of 15

Table 1: Methods compared in the main sparse and supporting studies. Software provenance, wrapper behavior, and reproducibility details appear in Online Resource 1 (§S5, S9).

Method	Role	Origin	Problem model	Execution behavior
LR-TA	internal seed	[5, 6]	sparse FAS / ordering	deterministic for fixed input and tie-breaking
WMSF-style seed	internal seed	adapted from [7, 23]	sparse FAS	deterministic for fixed input
IPSNS	proposed method	this study	sparse FAS refinement	stochastic through score-weighted SCC selection; deterministic destroy/repair prefixes conditional on the selected SCC
Borda / net score	calibration	[16]	ordering score rule	deterministic
Weighted Eades adaptation	adaptation	adapted from [12, 13]	sparse ordering heuristic	deterministic
Random multistart	calibration	standard multistart	ordering heuristic	stochastic
igraph Eades	external baseline	[17]	graph-library FAS heuristic	deterministic as evaluated
DRMacIver/FAS	external baseline	[18]	matrix ordering	evaluated executable shows run-to-run variation under internal initialization
Exact DP	exact validator	repository implementation	sparse FAS, $n \leq 20$	deterministic
HiGHS MIP	exact validator	[24] via `scipy.optimize.milp`	sparse-digraph MIP	deterministic under fixed settings

instances supports the time-capped MIP validation. Third, a parameter study splits 43 sparse instances into 18 tuning instances and 25 holdout instances; its purpose is only to check whether the chosen IPSNS defaults are defensible engineering choices.

The dense transfer benchmark is a 50-instance subset of LOLIB 2010 [19, 20], comprising 25 SGB, 10 IO, and 15 RandA1 instances. These are complete weighted ordering instances rather than sparse general digraphs. They are included to delimit regime dependence: sparse-benchmark success should not be interpreted automatically as dense-ordering dominance.

5.2 Methods compared

IPSNS is the primary method under study. It starts from the better of two internal seeds and applies incumbent-protected SCC-local refinement. The two seeds are LR-TA and a WMSF-style adaptation of the weighted minimal/stable line [7, 23]. They are retained in the tables because they clarify what the refinement adds and because the IPSNS non-worsening statement is relative to those seeds.

External and calibration baselines are chosen to cover accessible comparison classes rather than to exhaust the ranking literature. DRMacIver/FAS [18] is an external matrix-based ordering heuristic and the main external comparator in the sparse study. The evaluated implementation maintains a complete pairwise weight matrix: absent arcs are treated as zero-weight comparisons, sparse arc lists are expanded into this matrix for input, and the returned ordering is converted back to backward

Table 2: Main experimental studies and their accounting rules. Timeout rules, subset definitions, and traceability identifiers are documented in Online Resource 1.

Study	Purpose	Inst.	Reps	Reference / completion rule	Primary analysis
Primary sparse comparison	main sparse benchmark	97	1	all methods complete except DRMacIver/FAS (93/97); paired means use the 93-instance common subset	mean BW, best/tie counts, comparator-normalized reduction
Exact-validation study	small-instance certification	57	1	DP optimum for $n > 0$ standard instances	optimum-normalized gap
MIP-validation study	medium-instance certification	15	1	7 proven-optimal within 120 s; 8 time-limited	gap when certified
Holdout parameter study	default selection	43	5	18 tuning + 25 holdout instances	holdout median final BW
Repeated-run comparison	typical repeated-run behavior	93	20	common sparse subset completed by IPSNS and DRMacIver/FAS	paired per-instance medians and tests
Dense transfer study	complete-ordering boundary	50	1	single run per method	mean BW and best/tie counts

Table 3: Benchmark structural characteristics. Density is $m/[n(n-1)]$ for directed simple graphs after parallel-arc aggregation. Sparse statistics use the 97 standard non-negative `graph-benchmarks` instances; LOLIB statistics use the 50-instance transfer subset (25 SGB, 10 IO, 15 RandA1).

Characteristic	Sparse standard benchmark	Dense LOLIB transfer
Instances	97	50
n range / median	1–24,255 / 10	50–150 / 94
m range / median	1–34,876 / 21	complete pairwise orderings
Density median (Q1–Q3)	0.114 (0.003–0.583)	≈ 1 (complete order)
DAG fraction	4/97	0/50
Largest-SCC fraction (median)	0.64	0.99
Nontrivial SCC count (median)	11	1
Fraction in nontrivial SCCs (median)	0.78	0.99
Input structure	sparse general digraphs with localized cyclic structure	complete weighted ordering instances with globally coupled pairwise structure

weight on the original sparse digraph. The executable starts from a random initialization and iteratively applies a principal pairwise-ordering improvement operator until a local stopping criterion is met; repeated calls therefore can differ even on a fixed input. The repository wrapper invokes the binary with a 300 s timeout and reports interface failures explicitly. On DAG inputs, the evaluated binary returns an empty-tournament error that the wrapper records as an incomplete run rather than a zero-weight solution. The repeated-run study is motivated by this run-to-run variation. The python-igraph interface [17] supplies a graph-library Eades-style baseline.

Weighted Eades, Borda/net-score, and random multistart are included as transparent local calibrators. Exact DP is used only on the 57-instance validation subset, and HiGHS MIP is used only for the medium-instance certification study.

5.3 Evaluation metrics and accounting rules

Backward weight (BW) is the primary metric throughout. Every method is evaluated under the same BW objective on the original aggregated arc weights, regardless of whether the method internally operates on an ordering, a removed arc set, or a matrix representation.

The sparse and dense comparison tables report *Best*, meaning the number of instances on which a method attains the minimum observed BW among the evaluated methods. Ties are credited to all tied methods. *IPSNS comparator-normalized reduction* is the mean of per-instance values $(X - \text{IPSNS})/X \times 100$, where X is the comparator BW on the relevant common subset. Instances with $X = 0$ contribute 0%; incomplete comparator runs are excluded from paired means. Exact-validation tables report optimal-match counts and the mean optimum-normalized gap

$$\frac{\text{BW}_{\text{heuristic}} - \text{BW}_{\text{opt}}}{\text{BW}_{\text{opt}}} \times 100,$$

with zero-optimum instances handled separately by assigning gap 0 when the heuristic also attains optimum 0. For the dense LOLIB transfer study, the reported mean instance-level IPSNS-normalized gap uses $(\text{BW}_{\text{DR}} - \text{BW}_{\text{IPSNS}})/\text{BW}_{\text{IPSNS}} \times 100$ averaged over instances with $\text{BW}_{\text{IPSNS}} > 0$. Runtime is reported only as contextual wall-clock information and is not normalized across different external implementations.

Common-subset accounting is enforced wherever completion differs across methods. In the primary sparse comparison, DRMacIver/FAS completes 93 of the 97 standard instances; Panel A of Table 5 reports all methods on that common 93-instance subset, while Panel B records completion availability. The repeated-run study uses the same 93-instance common subset. The MIP study reports certified gaps only on the 7 instances proved optimal within the time limit and treats the remaining 8 time-limited cases as incomplete reference evidence rather than heuristic failures.

5.4 Statistical treatment

Paired comparisons use the instance as the statistical unit. For the primary sparse and repeated-run studies, the paired outcome is the per-instance difference in BW between two methods on the relevant common subset. The repeated-run analysis compares per-instance medians over 20 runs with tie tolerance 10^{-9}. Two-sided Wilcoxon signed-rank and sign tests are applied to the paired median differences. The Wilcoxon call uses SciPy 1.17.1 `scipy.stats.wilcoxon` with `alternative="two-sided"`, `zero_method="wilcox"`, and the default `method="auto"` (exact when feasible, otherwise asymptotic); the archived output records the test statistic and p-value but not the branch selected by `auto`. The sign test excludes ties from the win/loss count. For the IPSNS versus DRMacIver/FAS repeated-run comparison this leaves 38 nonzero

Table 4: IPSNS engineering defaults verified from `src/mwfas/ipsns.py`. These are holdout-supported defaults, not claims of universal optimality.

Parameter	Default	Role
Iteration budget T	400	outer destroy-and-repair iterations
Top-K SCC pool	15	score-weighted SCC candidate pool
Destroy add-back fraction	0.30	heavy removed internal arcs reactivated
Destroy removal fraction	0.02	light active internal arcs removed
Numerical tolerance τ	10^{-12}	tight-weight deactivation threshold
Seed policy	better of LR-TA and WMSF	initial incumbent selection
SCC-selection mode	weighted top-K	score-proportional random SCC choice

paired median differences and 55 exact ties. Bootstrap confidence intervals are paired, use seed 42, and use 10,000 resamples. Comparator-normalized reduction in the repeated-run study is computed from per-instance medians using

$$(\mathrm{DR} - \mathrm{IPSNS})/\mathrm{DR} \times 100. \tag{11}$$

5.5 Repeated-run and holdout protocols

The repeated-run comparison is a confirmatory study on the 93-instance common sparse subset. Each instance receives 20 IPSNS runs with explicit integer seeds and 20 DRMacIver/FAS repetitions through the same conversion and BW recomputation pipeline used in the main sparse comparison. The primary analysis compares per-instance median BW values with tie tolerance 10^{-9}. The protocol and analysis plan were fixed before examining aggregate outcomes. Reported summaries include wins, ties, losses, paired mean and median differences, comparator-normalized reduction, test results, effect size, and bootstrap intervals. Full per-instance summaries appear in Online Resource 1.

The holdout study is intentionally modest. It is not a main benchmark comparison but a conservative check on engineering defaults. Across 18 tuning and 25 holdout sparse instances, six parameter configurations were each run with five seeds per instance. The holdout aggregate did not separate the tested defaults on median final BW, so the chosen IPSNS settings are interpreted as holdout-supported defaults rather than tuned optima.

5.6 Implementation environment and reproducibility scope

All reported numbers are taken from tracked experiment outputs and regenerated manuscript tables. The runs used a Linux workstation with an Intel Core i7-12700K CPU, 62 GiB RAM, Ubuntu 24.04, and Python 3.12. External tools were invoked through documented interfaces; failures or incompletions were retained explicitly in the summaries. Online Resource 1 supplies the deferred protocols, configurations, processed outputs, proofs, and regeneration scripts. Full reruns additionally require the documented external data, binaries, and computational resources.

Table 5: Primary sparse comparison on the 97 standard nonnegative instances. Panel A reports all methods on the 93-instance subset completed by DRMacIver/FAS; Panel B reports completion availability. BW denotes backward weight (lower is better). Best counts every method attaining the minimum observed BW on an instance, with ties credited to all tied methods. IPSNS reduction is the mean comparator-normalized reduction $(X - \text{IPSNS})/X \times 100$ over Panel A instances with $X > 0$; zero-zero pairs contribute 0%.

Panel A: common 93-instance subset

Method	Mean BW	Median BW	Best	Mean inst.-level IPSNS red. (%)
IPSNS (ours)	29,586	5,118	92	0.00
LR-TA (ours)	30,197	5,118	78	0.70
WMSF seed	31,535	5,118	59	1.96
DRMacIver/FAS	53,173	5,649	56	21.14
igraph Eades	64,438	6,120	38	28.97
Weighted Eades	67,418	6,343	40	28.89
Borda net score	270,430	12,394	25	54.67
Random multistart	597,760	8,860	40	48.72

Panel B: completion on all 97 standard instances

Method	Att.	Comp.	Incompletion notes
IPSNS (ours)	97	97/97	—
LR-TA (ours)	97	97/97	—
WMSF seed	97	97/97	—
DRMacIver/FAS	97	93/97	`gr00`, `gr7`: interface failure on DAG inputs; `s38417`, `s38584`: timeout
igraph Eades	97	97/97	—
Weighted Eades	97	97/97	—
Borda net score	97	97/97	—
Random multistart	97	97/97	—

6 Computational results

The results are organized around the paper's main empirical questions. The primary question is how IPSNS performs on the standard sparse nonnegative benchmark. The supporting questions are whether that performance survives exact validation, how repeated-run behavior compares with the strongest external executable, what the ablation evidence says about the integrated design, and where the sparse-graph advantage stops transferring.

6.1 Primary sparse comparison

Table 5 reports the main comparison on the 97 standard nonnegative sparse instances.

IPSNS attains the minimum observed BW among the evaluated methods on 96 of the 97 instances when ties are credited to every tied method, but only 14 of the 97 are strict IPSNS unique bests. LR-TA and the WMSF-style seed are the strongest internal comparators, yet both trail IPSNS on the common 93-instance subset in mean BW and best-count frequency. Among external methods, DRMacIver/FAS is the strongest comparator; on the common subset IPSNS reduces BW by 21.14% relative

Table 6: Paired IPSNS contribution beyond the better seed on all 97 standard nonnegative instances. Seed BW is $\min\{\mathrm{BW}_{\mathrm{LR-TA}}, \mathrm{BW}_{\mathrm{WMSF}}\}$ on each instance.

Statistic	Value
Strict improvements	14
Ties	83
Regressions	0
Mean absolute BW improvement	571
Median absolute BW improvement	0
Mean relative improvement (%)	0.42
Median relative improvement (%)	0.0
Maximum absolute improvement	35,707
Instances with ≥ 1 accepted improving IPSNS move (EXP10, 93 common)	12/93
Mean accepted-move count / time-to-best	not logged in primary runs

to DRMacIver/FAS under the comparator-normalized convention. The incompletions remain explicit: DRMacIver/FAS completes 93 of the 97 instances because `gr00` and `gr7` are DAG inputs rejected by the evaluated binary through the conversion interface, and `s38417` and `s38584` time out in the external wrapper.

Table 6 quantifies what IPSNS adds beyond seeding on all 97 standard instances. Relative to the better of LR-TA and the WMSF-style seed, IPSNS yields 14 strict improvements, 83 ties, and 0 regressions, with mean absolute improvement 571 BW units (0.42% relative) and maximum improvement 35,707 on `rd_big`. On the full 105-instance sparse benchmark, IPSNS records zero incumbent violations relative to the better seed. LR-TA remains the low-latency option, with mean runtime 0.08 seconds per instance on the 97-instance standard set, but IPSNS is the method that attains the 96-of-97 minimum-observed result.

Single-run paired outcomes against DRMacIver/FAS on the 93 common standard instances are 37 wins, 55 ties, and 1 loss for IPSNS. Because those counts are tie-heavy and the external executable varies across repetitions, the more informative stability claim comes from the repeated-run analysis in Section 6.3. Full sparse paired-test tables for all baselines are deferred to Online Resource 1.

6.2 Exact validation, MIP checks, and holdout confirmation

Table 7 gives the exact small-instance validation results.

IPSNS matches the exact dynamic-programming optimum on 56 of the 57 standard nonnegative instances with $0 < n \leq 20$. LR-TA matches on 55 and the WMSF-style seed on 51. The mean optimum-normalized gap is 0.0031% for IPSNS; the sole miss on `r20_60` has a 3-unit BW gap and an optimum-normalized gap of 0.178%. The correct interpretation is empirical near-optimality on the validated subset, not a new optimality guarantee for larger graphs.

The time-capped MIP study extends that picture slightly beyond the DP regime and overlaps the DP-validated subset on small instances such as `r20_60`; it additionally includes medium instances beyond the DP size limit. On 15 selected medium instances, HiGHS certifies optimality on 7 and times out on 8. IPSNS matches the certified optimum on 6 of the 7 solved cases; again, the sole mismatch is `r20_60`, with IPSNS BW 1688 versus certified MIP optimum 1685 and an optimum-normalized gap of

Table 7: Exact validation on the 57 standard nonnegative instances with $n > 0$ and $n \leq 20$. Mean gap is the arithmetic mean of per-instance optimum-normalized gaps $(\mathrm{BW}_{\mathrm{heuristic}} - \mathrm{BW}_{\mathrm{opt}})/\mathrm{BW}_{\mathrm{opt}} \times 100$; zero-optimum instances contribute 0%.

Method	Optimal	Optimality rate	Mean opt-norm gap (%)
IPSNS (ours)	56/57	98.2%	0.0031
LR-TA (ours)	55/57	96.5%	0.35
WMSF seed	51/57	89.5%	0.48

This table supports empirical near-optimality on small instances only; it does not imply a general approximation guarantee. The only IPSNS near-miss is `r20_60`, where the BW gap is 3 and the optimum-normalized gap is 0.178%.

0.178%. The remaining 8 cases are reported only as incomplete certification attempts. Detailed per-instance MIP results are deferred to Online Resource 1.

The holdout parameter study is intentionally conservative. Across 18 tuning and 25 holdout instances, six IPSNS configurations produced the same holdout aggregate median final BW under the five-seed protocol. The paper therefore keeps the default configuration in Table 4 as a reasonable engineering choice rather than claiming that the selected parameters are uniquely optimal. On the 10-instance ablation subset, 50- and 400-iteration variants have equal mean BW; 400 iterations is retained as the conservative default because holdout medians did not favor a smaller budget and the extra iterations did not worsen quality, although they increase runtime on harder instances.

6.3 Repeated-run comparison with DRMacIver/FAS

On the 93-instance common sparse subset, each method was run 20 times per instance and compared through per-instance median backward weight (BW). IPSNS records 38 wins, 55 ties, and no losses against the evaluated DRMacIver/FAS executable, so the effective nonzero sample size is 38. The mean comparator-normalized reduction is 21.60%, while the median reduction is 0.0% because ties dominate the paired medians.

The paired differences are significant under both a two-sided Wilcoxon signed-rank test ($W = 0$, $p < 0.001$) and a two-sided sign test ($p < 0.001$). Cohen's d_z is -0.31, and the paired 95% bootstrap confidence interval for the mean BW difference is $[-41{,}787, -10{,}718]$. Thus, the typical common instance is a tie, but the non-tied outcomes are strongly asymmetric in favor of IPSNS, and several large improvements drive the positive mean reduction.

Run-level logs show that 87.1% of IPSNS runs accept no improving move and only 12 of the 93 common instances ever improve over the best seed under the tested configuration. IPSNS nevertheless shows one distinct BW value on all 93 instances across 20 seeds. Zero objective variance therefore indicates stable final objective values under the tested configuration; it does not by itself establish that search trajectories or final orderings are identical, and the randomized SCC-selection step had little effect on final BW in this study. DRMacIver/FAS, by contrast, shows run-to-run objective

Table 8: Dense LOLIB transfer study on 50 complete weighted ordering instances ($n = 94$ median; complete pairwise structure). Lower BW is better. Best counts every method attaining the minimum observed BW, with ties credited to all tied methods.

Method	Mean BW	Best	Mean runtime (s)
DRMacIver/FAS	571,687	45/50	1.01
IPSNS (ours)	582,354	5/50	37.92
LR-TA (ours)	582,948	2/50	0.22
WMSF seed	585,926	1/50	0.19
igraph Eades	659,570	0/50	0.01
Weighted Eades	675,235	0/50	0.01
Random multistart	883,664	0/50	0.02
Borda net score	1,066,045	0/50	0.00

variation on 40 of the 93 instances, which is why a single execution is not always representative for that executable.

6.4 Ablation evidence

The ablation study on the 10-instance representative sparse subset supports two points. First, topological add-back is the dominant seed-side design choice: removing it increases the subset mean BW from 4271.5 to 4525.1, a 5.94% ratio-of-means increase relative to LR-TA. Second, IPSNS reduces the subset mean BW from 4271.5 to 4239.2, a further 0.76% ratio-of-means reduction after seeding, and the 50- and 400-iteration variants have equal mean BW on this subset. That second gain is smaller, but it is achieved after a strong seed and under strict non-worsening. The full ablation table and secondary controls are moved to Online Resource 1.

6.5 Dense LOLIB boundary

Table 8 reports the dense transfer test.

In this single-run transfer study, the evaluated DRMacIver/FAS implementation outperformed IPSNS on the selected 50 LOLIB instances: minimum observed BW on 45 of 50 instances. The mean instance-level IPSNS-normalized gap $(\mathrm{BW}_{\mathrm{DR}} - \mathrm{BW}_{\mathrm{IPSNS}})/\mathrm{BW}_{\mathrm{IPSNS}} \times 100$ was -3.88%, indicating lower DRMacIver/FAS backward weight under that per-instance convention; the corresponding ratio of aggregate mean BW values was 571,687 versus 582,354. IPSNS remains second overall, but the selected complete-ordering benchmark and evaluated implementations favored DRMacIver/FAS in this transfer experiment.

This outcome is consistent with the problem structure rather than contradictory to the sparse story. IPSNS is built around sparse weighted-digraph seeding and SCC-local repair. LOLIB consists of complete weighted ordering instances with globally coupled cyclic structure, so the evaluated DRMacIver/FAS implementation is better aligned with the input model in this transfer experiment. The LOLIB test therefore serves as an explicit scope boundary for the main sparse-graph claim.

7 Discussion

The sparse-benchmark results fit the intended mechanism of IPSNS. LR-TA and the WMSF-style seed already produce strong feasible orderings, but they do so through different biases: LR-TA emphasizes cycle reduction and heavy-first repair, while the WMSF-style seed emphasizes ordered removals and stabilization. IPSNS inherits the better of those two constructions and then spends its search budget only where backward weight still remains concentrated.

That focus matters on sparse graphs because the difficult part of the objective is often localized. Large acyclic or weakly cyclic regions do not justify broad perturbation. The ablation evidence is consistent with that interpretation: most of the gain comes from engineering a strong seed, and IPSNS then improves the harder residue without ever sacrificing the better seed. The gains can be numerically modest because the starting point is already strong, but consistency matters here more than large changes on every instance.

The repeated-run comparison sharpens this interpretation. The median outcomes are tie-heavy, so the typical common instance does not show a dramatic change between IPSNS and DRMacIver/FAS. But the non-tied outcomes are one-sided in favor of IPSNS, and the aggregate mean comparator-normalized reduction remains large. That is the profile one would expect from a protected refinement layer: many instances are already effectively solved by the seed or are insensitive to local refinement, while a smaller subset carries the material gains.

Three limitations deserve emphasis.

- **Nonnegative scope.** Negative-weight instances are excluded because the local-ratio and incumbent-protection statements are developed for nonnegative weights.
- **Dense transfer boundary.** The single-run LOLIB study shows that the selected complete-ordering benchmark and evaluated implementations favored DRMacIver/FAS; this should not be read as a statement about all matrix-based ordering methods.
- **Stochastic interpretation.** Zero observed IPSNS objective variance across 20 seeds documents stable final objective values for the tested configuration, not mathematical determinism or identical search trajectories.

Within those limits, the main conclusion remains intact: incumbent-protected SCC-local refinement is a useful algorithm-engineering strategy for sparse nonnegative weighted digraphs. It builds on attributed seed methods, improves them selectively, and reports clearly where that advantage does not transfer.

8 Conclusions

This paper studied MWFAS in the regime of sparse nonnegative weighted digraphs. The central contribution is IPSNS, an incumbent-protected SCC-local destroy-and-repair heuristic that starts from the better of two attributed seeds, concentrates refinement on SCCs that still contribute backward weight, and accepts a candidate only when the global objective strictly improves. The seeds themselves are supporting components: a local-ratio topological add-back construction rooted in prior local-ratio

work and earlier author work, and a WMSF-style seed adapted from Cavallaro and Cutello's weighted minimal/stable line.

The main empirical result is deliberately scoped. On the primary sparse benchmark, IPSNS attains the minimum observed backward weight among the evaluated methods on 96 of 97 standard nonnegative instances, with ties credited to every tied method (14 strict wins). Relative to the better seed on the same instances, IPSNS yields 14 strict improvements, 83 ties, and 0 regressions. On the exact-validation subset, it matches the dynamic-programming optimum on 56 of 57 instances, with the only near-miss again occurring on `r20_60`. On the 93-instance common sparse subset, repeated-run per-instance medians over 20 runs give 38 wins, 55 ties, and 0 losses versus the evaluated DRMacIver/FAS executable. These results support the interpretation of IPSNS as a consistent refinement layer rather than a one-off benchmark outlier.

The supporting studies also mark the method's boundary. Heavy-first add-back is the dominant seed-side design choice, IPSNS adds a smaller but meaningful improvement after strong seeding, and the rollback rule ensures that this gain never comes at the cost of a worse final solution than the better seed. The single-run dense LOLIB transfer test then shows where the sparse advantage stops transferring: in that experiment the selected complete-ordering benchmark and evaluated DRMacIver/FAS implementation outperformed IPSNS on 45 of 50 instances. Future work should test whether denser or hybrid seed constructions can extend the same incumbent-protected refinement discipline beyond the sparse setting.

Statements and Declarations

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests

The author declares that there are no known competing financial or non-financial interests that could have appeared to influence the work reported in this paper.

Ethics approval

Not applicable. This study involves no human participants, human data, or animal subjects; it is a computational study on public graph benchmark instances.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Author contributions

Soroush Vahidi: Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Related manuscripts and prior author work

This manuscript is a revised and retargeted version of a manuscript previously submitted to *Computational Optimization and Applications* under the title "IPSNS for Minimum Weighted Feedback Arc Set on Sparse Digraphs"; that submission was declined, and the COAP submission is closed and not under active consideration anywhere. Section 2.4 describes the relationship to the public preprint [6] and explains how the LR-TA and WMSF-style components are attributed. The author also has a separate manuscript, "Learning-Free Ranking from Pairwise Comparisons via Feedback-Arc-Set Pruning and Add-Back," under review at *The Journal of Supercomputing*; an overlap assessment between that manuscript and the present one is in progress and will be completed and disclosed to the editor before this manuscript is submitted. No other substantially overlapping manuscript is knowingly under consideration elsewhere at the time of this submission.

Data and code availability

The source code, experiment outputs, Online Resource 1, and submission bundle are publicly available from the GitHub repository https://github.com/SoroushVahidi/minimum-weighted-fas-heuristics, which is public at the time of submission. The SN Computer Science manuscript PDF, source snapshot, and upload bundle are served from `paper_sncs/submission/sncs_initial/` on the repository main branch; the scientific source snapshot recorded in `online_resource_1/provenance/source_commit.txt` identifies the exact commit used to build the accompanying artifact. Online Resource 1 is provided as `online_resource_1/Online_Resource_1.pdf` and as the upload ZIP in the same `sncs_initial/` directory. Public benchmark families are cited in the manuscript: sparse instances from `graph-benchmarks` and dense LOLIB 2010 instances. External-tool versions and acquisition procedures are documented in Online Resource 1 (§S5, S8–S9). No Zenodo DOI is assigned at the time of submission.

Reproducing the full benchmark runs additionally requires the documented external data, third-party tools, and computational resources described in Online Resource 1. The package is intended to make the reported evidence traceable without requiring every reader to rerun the most expensive computations.

Generative AI and AI-assisted technologies

During the preparation of this work, the author used AI-assisted tools, including ChatGPT, Codex, Claude, and Perplexity AI, to support literature exploration, organization of material, language editing, and coding assistance. These tools did not independently select scientific conclusions, create experimental observations, fabricate

or alter data, certify proofs, determine acceptance or rejection decisions, or supply unverified references as final citations. The author reviewed and edited all outputs, checked references against primary sources, reviewed and tested code changes, and recomputed or validated numerical claims. The author assumes full responsibility for the content of the submitted manuscript.

References

[1] Karp, R.M.: Reducibility among combinatorial problems. In: Complexity of Computer Computations, pp. 85–103. Springer, Boston, MA (1972). https://doi.org/10.1007/978-1-4684-2001-2_9

[2] Flood, M.M.: Exact and heuristic algorithms for the weighted feedback arc set problem: A special case of the skew-symmetric quadratic assignment problem. Networks **20**(1), 1–23 (1990) https://doi.org/10.1002/net.3230200102

[3] Ailon, N., Charikar, M., Newman, A.: Aggregating inconsistent information: Ranking and clustering. Journal of the ACM **55**(5), 1–27 (2008) https://doi.org/10.1145/1411509.1411513

[4] Baharev, A., Schichl, H., Neumaier, A., Achterberg, T.: An exact method for the minimum feedback arc set problem. ACM Journal of Experimental Algorithmics **26**, 1–28 (2021) https://doi.org/10.1145/3446429

[5] Demetrescu, C., Finocchi, I.: Combinatorial algorithms for feedback problems in directed graphs. Information Processing Letters **86**(3), 129–136 (2003) https://doi.org/10.1016/S0020-0190(02)00491-X

[6] Vahidi, S., Koutis, I.: Ranking from Pairwise Comparisons as Minimum Weighted Feedback Arc Set (2024). https://doi.org/10.48550/arXiv.2412.16181 . https://arxiv.org/abs/2412.16181

[7] Cavallaro, C., Cutello, V.: An efficient heuristic algorithm to compute minimal and stable weighted feedback arc sets. In: Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE), pp. 84–87 (2025). https://doi.org/10.18293/SEKE2025-049

[8] Lempel, A., Cederbaum, I.: Minimum feedback arc and vertex sets of a directed graph. IEEE Transactions on Circuit Theory **13**(4), 399–403 (1966) https://doi.org/10.1109/TCT.1966.1082620

[9] Bar-Yehuda, R., Geiger, D., Naor, J.S., Roth, R.M.: Approximation algorithms for the feedback vertex set problem with applications to constraint satisfaction and bayesian inference. SIAM Journal on Computing **27**(4), 942–959 (1998) https://doi.org/10.1137/S0097539796305109

[10] Even, G., Naor, J.S., Schieber, B., Sudan, M.: Approximating minimum feedback

sets and multicuts in directed graphs. Algorithmica **20**(2), 151–174 (1998) https://doi.org/10.1007/PL00009191

[11] Hecht, M., Gonciarz, K., Horvát, S.: Tight localizations of feedback sets. ACM Journal of Experimental Algorithmics **26**, 1–19 (2021) https://doi.org/10.1145/3447652

[12] Eades, P., Lin, X., Smyth, W.F.: A fast and effective heuristic for the feedback arc set problem. Information Processing Letters **47**(6), 319–323 (1993) https://doi.org/10.1016/0020-0190(93)90079-O

[13] Eades, P., Lin, X.: A heuristic for the feedback arc set problem. Australasian Journal of Combinatorics **12**, 15–25 (1995)

[14] Brandenburg, F.J., Hanauer, K.: Sorting heuristics for the feedback arc set problem. In: WALCOM: Algorithms and Computation. Lecture Notes in Computer Science, vol. 7748, pp. 1–12. Springer, Berlin, Heidelberg (2013)

[15] Simpson, M., Srinivasan, V., Thomo, A.: Efficient computation of feedback arc set at web-scale. Proceedings of the VLDB Endowment **10**(3), 133–144 (2016) https://doi.org/10.14778/3021924.3021930

[16] Coppersmith, D., Fleischer, L.K., Rudra, A.: Ordering by weighted number of wins gives a good ranking for weighted tournaments. ACM Transactions on Algorithms **6**(3), 1–13 (2010) https://doi.org/10.1145/1798596.1798608

[17] python-igraph developers: python-igraph documentation: `Graph.feedback_arc_set`. https://python.igraph.org/en/stable/api/igraph.Graph.html. Documentation for the weighted feedback arc set API; accessed 2026-06-06 (2026)

[18] DRMacIver: Feedback-Arc-Set. https://github.com/DRMacIver/Feedback-Arc-Set. Software repository; experiment commit 16ff24a92fde886e58819180a9fe686e60991c5c; accessed 2026-06-06 (2026)

[19] Marti, R., Reinelt, G., Duarte, A.: A benchmark library and a comparison of heuristic methods for the linear ordering problem. Computational Optimization and Applications **51**(3), 1297–1317 (2012) https://doi.org/10.1007/s10589-010-9384-9

[20] Marti, R.: LOLIB: Linear Ordering Problem Library. https://www.uv.es/~rmarti/paper/lop.html. Official library page; accessed 2026-06-06 (2010)

[21] Fomin, F.V., Lokshtanov, D., Raman, V., Saurabh, S.: Fast local search algorithm for weighted feedback arc set in tournaments. Proceedings of the AAAI Conference on Artificial Intelligence **24**(1), 65–70 (2010) https://doi.org/10.1609/aaai.v24i1.7557

[22] Ostovari, M., Zarei, A.: A better LP rounding for feedback arc set on tournaments. Theoretical Computer Science **1015**, 114768 (2024) https://doi.org/10.1016/j.tcs.2024.114768

[23] Cavallaro, C., Cutello, V., Pavone, M.: Efficient heuristics to compute minimal and stable feedback arc sets. Journal of Combinatorial Optimization **48** (2024) https://doi.org/10.1007/s10878-024-01209-8

[24] Huangfu, Q., Hall, J.: Parallelizing the dual revised simplex method. Mathematical Programming Computation **10**(1), 119–142 (2018) https://doi.org/10.1007/s12532-017-0130-5

[25] Ali Dasdan: graph-benchmarks: Directed Graph Benchmarks in DIMACS Graph Format. https://github.com/alidasdan/graph-benchmarks. Dataset repository; accessed 2026-06-06 (2026)